

ระเบียบวิธีการทำโครงการ ทักษะที่ใช้ ฮาร์ดแวร์ หน่วยประมวลผลกลาง (CPU) ขั้นต่ำ 2.5 GHz และ 8 cores หน่วยความจำหลัก (RAM) 32 GB พื้นที่เก็บข้อมูล 30 GB หน่วยประมวลผลกราฟิก (GPU) ขนาด 16 GB VRAM ซอฟต์แวร์และเครื่องมือ Python รุ่น 3.13 ONNX Runtime รุ่นล่าสุด CTranslate2 รุ่นล่าสุด MetalNLLB รุ่นล่าสุด Tauri Framework รุ่น v2 Rust toolchain รุ่น stable Git รุ่นล่าสุด แบบจำลองต้นแบบ โมเดล PP-OCRV5-mobile โมเดล ThaiTrOCR โมเดล NLLB-200-distilled-600M โมเดล NLLB-200-distilled-1.3B การเตรียมข้อมูล สร้างภาพสังเคราะห์ สำหรับ OCR เรียกใช้โปรแกรม SynthTIGER บนเครื่องคอมพิวเตอร์เพื่อเริ่มต้นกระบวนการสร้างภาพ ระบุแหล่งข้อมูล: ดึงข้อความภาษาไทยและภาษาอังกฤษจาก Wikipedia แบบสุ่ม โดยกำหนดสัดส่วน ภาษาไทยร้อยละ 50 และภาษาอังกฤษร้อยละ 50 กำหนดแบบอักษร (font) จำนวน 5 ชนิด ได้แก่ TH Sarabun PSK, Noto Sans Thai, Kanit, Prompt และ Sriracha ตั้งค่าขนาดภาพเท่ากับ 512x512 px ประกอบด้วยความกว้าง 512 พิกเซลและความสูง 512 พิกเซล กำหนดการเพิ่ม สัญญาณรบกวน (noise) แบบเกาส์ (Gaussian) โดยกำหนดค่า sigma ตั้งแต่ 10 ถึง 30 กำหนด จำนวนภาพที่ต้องการสร้างเท่ากับ 10,000 ภาพ ดำเนินการสร้างภาพทั้งหมดด้วย SynthTIGER ตาม ค่าที่กำหนดในขั้นตอนที่ 2-6 ตรวจสอบผลลัพธ์: สุ่มเปิดภาพตัวอย่าง 5 ภาพ เพื่อยืนยันว่าภาพที่สร้างขึ้นมีความถูกต้องและตรงตามพารามิเตอร์ที่กำหนดไว้ บันทึกภาพสังเคราะห์ทั้งหมด 10,000 ภาพลงในโฟลเดอร์ data/synthetic เก็บภาพหน้าจอจริงสำหรับ OCR จับภาพหน้าจอจากระบบปฏิบัติการ (native screen capture) หรือโปรแกรมจับภาพหน้าจอใด ๆ บนระบบปฏิบัติการเป้าหมาย (Windows, macOS หรือ Linux) เลือกซอฟต์แวร์เป้าหมายที่ใช้งานทั่วไปซึ่งแสดงข้อความภาษาไทย และภาษาอังกฤษปะปนกัน โดยครอบคลุมลักษณะข้อความที่แตกต่างกัน ได้แก่ ความหนาแน่นของ ข้อความทั้งระดับสูงและระดับต่ำ ส่วนติดต่อผู้ใช้ทั้งแบบเทคนิคและแบบทั่วไป ข้อความหลายภาษา และข้อความบนพื้นหลังที่มีคอนทราสต์ต่างกัน จับภาพหน้าจอของซอฟต์แวร์แต่ละโปรแกรมจนรวม ทั้งสิ้น 500 ภาพ โดยกระจายจำนวนภาพให้ครอบคลุมลักษณะข้อความที่หลากหลายข้างต้น สร้างคำตอบอ้างอิง (ground truth) สำหรับภาพทั้งหมดด้วยการพิมพ์ข้อความในภาพซ้ำด้วยมือ ตรวจสอบ ความถูกต้องของตัวอักษรทุกตัว ตรวจสอบคำตอบอ้างอิงแบบสุ่มร้อยละ 10 ของภาพทั้งหมดโดยผู้ ตรวจสอบคนที่สอง เพื่อลดความผิดพลาดจากการพิมพ์ บันทึกภาพหน้าจอจริงพร้อมคำตอบอ้างอิงลงในโฟลเดอร์ data/real ดาวน์โหลดชุดทดสอบสาธารณะ ดาวน์โหลดชุดข้อมูล ThaiOCRBench ซึ่ง ประกอบด้วยภาพ 2,808 ภาพจาก 13 การกึ่งการรู้จำข้อความ ดาวน์โหลดชุดข้อมูล thai-ocr-evaluation ซึ่งประกอบด้วยภาพ 104 ภาพ ตรวจสอบจำนวนไฟล์และโครงสร้างโฟลเดอร์ของทั้งสอง ชุดข้อมูลให้ตรงตามเอกสารอ้างอิงต้นทาง แบ่งข้อมูล รวมภาพสังเคราะห์ 10,000 ภาพและภาพจริง 500 ภาพเข้าเป็นชุดข้อมูลเดียวกัน รวมทั้งสิ้น 10,500 ภาพ ใช้เทคนิคการแบ่งข้อมูลแบบแบ่งตาม ประเภทของภาพ (stratified split) ตามวิธีของ Raschka โดยกำหนดสัดส่วน: ชุดฝึก (training set) ร้อยละ 70 ชุดตรวจสอบ (validation set) ร้อยละ 15 และชุดทดสอบ (test set) ร้อยละ 15 ตรวจสอบจำนวนข้อมูลในแต่ละชุดเพื่อยืนยันสัดส่วน: ชุดฝึก 7,350 ภาพ ชุดตรวจสอบ 1,575 ภาพ และชุดทดสอบ 1,575 ภาพ จัดเก็บข้อมูลทั้งสามชุดในโฟลเดอร์แยกกัน: data/train, data/val และ data/test เตรียมข้อมูลคู่ภาษา ดาวน์โหลดชุดข้อมูล SCB-MT-EN-TH ซึ่งประกอบด้วยคู่ประโยค ภาษาไทย-ภาษาอังกฤษจำนวน 1,000,000 คู่ สุ่มเลือกคู่ประโยคจำนวน 100,000 คู่จากชุดข้อมูล ทั้งหมด ด้วยการสุ่มแบบแบ่งตามแหล่งที่มา (stratified random sampling) เพื่อรักษาสัดส่วนของ

ประเภทข้อความเดิม แบ่งข้อมูลที่สุ่มได้เป็นชุดฝึก 80,000 คู่ ชุดตรวจสอบ 10,000 คู่ และชุดทดสอบ 10,000 คู่ ดาวินโหลตชุดข้อมูล FLORES-200 ซึ่งประกอบด้วยประโยค 3,001 ประโยคใน 200 ภาษา รวมภาษาไทย สำหรับใช้เป็นชุดประเมิน (evaluation set) ตรวจสอบรูปแบบไฟล์ของทุกชุดข้อมูลให้อยู่ในรูปแบบ .txt ที่มีหนึ่งประโยคต่อหนึ่งบรรทัด การคัดเลือกแบบจำลอง คัดเลือกแบบจำลอง OCR เรียกใช้แบบจำลอง PP-OCRv5-mobile และ ThaiTrOCR จากที่เก็บแบบจำลอง (model repository) ที่ดาวินโหลตไว้ก่อนหน้านี้ กำหนดชุดค่าการทดลอง (experiment configuration): ใช้หน่วยประมวลผลกลาง (CPU) จำนวน 4 แกน (4 cores) และหน่วยความจำหลัก (RAM) ขนาด 8 GB ตรึงค่าตั้งต้น (seed) ของทุกไลบรารีที่เกี่ยวข้องให้มีค่าเท่ากันทุกการทดลอง เพื่อให้ผลลัพธ์ deterministic โหลดชุดข้อมูล ThaiOCRBench และ thai-ocr-evaluation เข้าสู่หน่วยความจำ ดำเนินการอนุมานผล (inference) บนชุดข้อมูลทั้งสอง ด้วยค่าการตั้งค่าที่กำหนดไว้ในขั้นตอนที่ 2 วัดค่า CER (Character Error Rate) จากผลการอนุมานผลเทียบกับคำตอบอ้างอิง และบันทึกเวลาคำนวณเฉลี่ยต่อภาพ (average computation time per image) มีหน่วยเป็นวินาที วัดซ้ำ 5 รอบ ต่อแบบจำลอง โดยใช้ข้อมูลชุดเดียวกันทุกครั้ง คำนวณค่าเฉลี่ย (mean) และส่วนเบี่ยงเบนมาตรฐาน (standard deviation) ของ CER และเวลาคำนวณจากทั้ง 5 รอบ เปรียบเทียบผลด้วยสถิติ paired t-test ที่ระดับนัยสำคัญ  $\alpha = 0.05$  เลือกแบบจำลองตามเกณฑ์: ถ้า ThaiTrOCR มีค่า CER  $\leq 0.10$  ให้เลือก ThaiTrOCR มิฉะนั้นให้เลือกแบบจำลองที่มีค่า CER ต่ำที่สุด คัดเลือกแบบจำลอง MT เรียกใช้แบบจำลอง NLLB-200-distilled-600M และ NLLB-200-distilled-1.3B จากที่เก็บแบบจำลองของ Hugging Face กำหนดชุดค่าการทดลอง: ใช้หน่วยประมวลผลกลาง (CPU) จำนวน 4 แกน (4 cores) และหน่วยความจำหลัก (RAM) ขนาด 8 GB โหลดชุดข้อมูล FLORES-200 ซึ่งประกอบด้วยประโยคภาษาไทย 3,001 ประโยค เข้าสู่หน่วยความจำ ดำเนินการอนุมานผล (inference) แบบทีละชุด โดยกำหนดขนาดชุดเท่ากับ 16 ประโยคต่อครั้ง วัดค่า BLEU ด้วยสคริปต์มาตรฐาน (sacrebleu) และวัดความเร็วในการแปลเป็นจำนวน tokens ต่อวินาที (tokens/s) วัดซ้ำ 5 รอบต่อแบบจำลอง โดยใช้ข้อมูลชุดเดียวกันทุกครั้ง คำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของ BLEU และ throughput จากทั้ง 5 รอบ เปรียบเทียบผลด้วยสถิติ paired t-test ที่ระดับนัยสำคัญ  $\alpha = 0.05$  เลือกแบบจำลองตามเกณฑ์: ถ้า NLLB-600M มีค่า BLEU  $> 35$  และ latency  $\leq 2$  วินาที ให้เลือก NLLB-600M มิฉะนั้นให้เลือก NLLB-1.3B การปรับเทียบแบบจำลอง ปรับเทียบแบบจำลอง OCR รวมชุดข้อมูล 3 ส่วนเข้าด้วยกัน ได้แก่ (ก) ภาพสังเคราะห์จาก SynthTIGER จำนวน 10,000 ภาพ (ข) ภาพหน้าจอจริง 500 ภาพ เพิ่มข้อมูล (data augmentation) ด้วยวิธีการดังนี้: การหมุนภาพแบบสุ่มในช่วง  $\pm 3^\circ$  การปรับความสว่างแบบสุ่มในช่วง  $\pm 20\%$  และการเติมสัญญาณรบกวนแบบเกาส์ (Gaussian noise) ค่า sigma = 5 ได้ชุดข้อมูลรวมสำหรับการปรับเทียบละเอียดทั้งสิ้น 15,000 ภาพ กำหนดวิธี Grid Search สำหรับค้นหาค่าตัวแปรปรับตั้งเหนือ (hyperparameters) ที่เหมาะสม โดยกำหนดช่วงค่า: อัตราการเรียนรู้ (learning rate) 5 ค่าในช่วง  $10^{-5}$  ถึง  $5 \times 10^{-5}$ , ขนาดชุด (batch size) 8, 16 และ 32, และรอบการฝึก (epoch) 10 ถึง 50 รอบ ตั้งค่าตัวปรับเหมาะ (optimizer) เป็น Adam ซึ่งเป็นอัลกอริทึมปรับเทียบที่ปรับอัตราการเรียนรู้ให้เหมาะสมอัตโนมัติ ตั้งค่าการหยุดก่อนกำหนด (early stopping): เมื่อค่า CER บนชุดตรวจสอบ (validation set) หากค่า CER ไม่ลดลงเป็นระยะเวลา 5 รอบการฝึกติดต่อกัน ให้หยุดการปรับเทียบก่อนครบรอบที่กำหนด ดำเนินการ Grid Search โดยฝึกแบบจำลองด้วยชุดค่าตัวแปรปรับตั้งเหนือทุกชุดที่เป็นไปได้ จากนั้น

บันทึกค่า CER บนชุดตรวจสอบของแต่ละชุดค่า เลือกชุดค่าตัวแปรปรับตั้งเหนือที่ให้ค่า CER ต่ำที่สุด บนชุดตรวจสอบเป็นค่าที่เหมาะสมที่สุด บันทึกแบบจำลองที่ปรับเทียบแล้วในรูปแบบ ONNX เพื่อใช้งานในการพัฒนาโปรแกรมประยุกต์ในขั้นตอนถัดไป ปรับเทียบแบบจำลอง MT เรียกใช้แบบจำลอง NLLB-200-distilled-600M จากที่เก็บ Hugging Face ใช้เทคนิค LoRA (Low-Rank Adaptation) สำหรับการปรับเทียบละเอียด โดยเพิ่มพารามิเตอร์ (weight) ที่สามารถฝึกได้  $\Delta W = BA$  เข้าไปในชั้น ที่กำหนด โดยคงน้ำหนักเดิมของแบบจำลองไว้ ตรึงน้ำหนักทั้งหมดของแบบจำลอง (freeze all weights) ไว้ไม่ให้เปลี่ยนแปลงระหว่างการปรับเทียบ ค้นหาตัวแปรปรับตั้งเหนือของ LoRA: อันดับ (rank)  $r \in \{8, 12, 16\}$  — กำหนดขนาดของเมทริกซ์  $B$  และ  $A$ , ค่า scaling factor  $\alpha \in \{16, 32, 48\}$  — กำหนดสัดส่วนของ  $\Delta W$  เมื่อรวมกับน้ำหนักเดิม, และ โมดูลเป้าหมาย (target modules): `q_proj` และ `v_proj` — ชั้นที่สอดคล้อง LoRA เข้าไป ตั้งค่าอัตราเรียนรู้ (learning rate) ในช่วง  $2 \times 10^{-4}$  ถึง  $5 \times 10^{-4}$  ตั้งค่ารอบการฝึก (epoch) ตั้งแต่ 3 ถึง 10 รอบ ตั้งค่าตัวปรับเหมาะ (optimizer) เป็น Adam ดำเนินการปรับเทียบและวัดค่า BLEU บนชุดข้อมูล FLORES-200 ทุกรอบการฝึก เพื่อติดตามการเปลี่ยนแปลง เลือกชุดค่าตัวแปรปรับตั้งเหนือที่ให้ค่า BLEU สูงที่สุดบนชุดประเมิน ใช้ NLLB-200-distilled-1.3B แบบศูนย์ตัวอย่าง (zero-shot configuration) โดยไม่ดำเนินการปรับเทียบเพิ่มเติม บันทึกแบบจำลองที่ปรับเทียบแล้วในรูปแบบ CTranslate2 เพื่อใช้งานในการพัฒนาโปรแกรมประยุกต์ในขั้นตอนถัดไป การพัฒนาโปรแกรมประยุกต์ สร้างโครงร่างโปรเจกต์ Tauri เรียกใช้คำสั่งสร้างโปรเจกต์ Tauri v2 ด้วยคำสั่ง `npm create tauri-app@latest` เพื่อสร้างโครงร่างโปรเจกต์เริ่มต้น กำหนดให้ Rust เป็นภาษาสำหรับส่วนหลังบ้าน (backend) สำหรับจัดการตรรกะของโปรแกรม กำหนดให้ Svelte เป็นเฟรมเวิร์กสำหรับส่วนหน้าบ้าน (frontend) สำหรับสร้างส่วนติดต่อผู้ใช้ ตรวจสอบโครงสร้างโฟลเดอร์ที่สร้างขึ้น: โฟลเดอร์ `src-tauri` สำหรับส่วนหลังบ้าน และโฟลเดอร์ `src` สำหรับส่วนหน้าบ้าน ติดตั้ง dependencies เพิ่มเติมที่จำเป็น: ไลบรารีสำหรับจับภาพหน้าจอ และ `onnxruntime` สำหรับเรียกใช้งานแบบจำลอง OCR ส่งออกแบบจำลองเป็นรูปแบบที่พร้อมใช้งาน ส่งออกแบบจำลอง OCR ที่ผ่านการคัดเลือกและปรับเทียบแล้วเป็นรูปแบบ ONNX ด้วยคำสั่งส่งออก (export) ของเฟรมเวิร์กที่ใช้ฝึก ส่งออกแบบจำลอง MT ที่ผ่านการคัดเลือกและปรับเทียบแล้วเป็นรูปแบบ CTranslate2 ซึ่งเป็นรูปแบบที่ออกแบบมาเฉพาะสำหรับการอนุมานผลบนหน่วยประมวลผลกลาง ทดสอบไฟล์แบบจำลองที่ส่งออกแล้ว: เรียกใช้งานไฟล์ ONNX และ CTranslate2 ด้วยโค้ดทดสอบสั้น ๆ เพื่อยืนยันว่าสามารถโหลดและอนุมานผลได้ถูกต้อง เชื่อมต่อฟังก์ชันจับภาพหน้าจอผ่านส่วนหลังบ้าน Rust เพิ่มไลบรารีจับภาพหน้าจอ (screen capture library) เป็น dependency ในไฟล์ `Cargo.toml` ของส่วนหลังบ้าน เขียนฟังก์ชันสำหรับจับภาพหน้าจอในภาษา Rust โดยใช้ API ของไลบรารีที่เลือก: ระบุพิกัด (x, y) และขนาด (width, height) ของพื้นที่ที่ต้องการจับภาพ ส่งภาพที่จับได้จากส่วนหลังบ้านไปยังสายงานประมวลผล OCR (OCR pipeline) เพื่อตรวจจับและรู้จำข้อความ จัดการข้อผิดพลาด (error handling): หากจับภาพไม่สำเร็จหรือภาพว่างเปล่า ให้ส่งข้อความแจ้งเตือนกลับไปยังส่วนหน้าบ้าน สร้างส่วนติดต่อผู้ใช้ด้วย Svelte สร้างปุ่ม “จับภาพหน้าจอ” บนส่วนติดต่อผู้ใช้ เมื่อผู้ใช้คลิกให้ส่งคำสั่งไปยังส่วนหลังบ้านเพื่อเริ่มจับภาพ แสดงผลข้อความที่ระบบ OCR ตรวจจับได้ในกรอบข้อความบนหน้าจอ แสดงผลการแปลภาษาในกรอบข้อความถัดจากผล OCR โดยให้อัปเดตอัตโนมัติเมื่อได้ผล

การแปล เพิ่มปุ่ม “คัดลอกผลลัพธ์” สำหรับคัดลอกทั้งข้อความต้นฉบับและข้อความที่แปลแล้วไปยังคลิปบอร์ด เพิ่มปุ่ม “บันทึก” สำหรับบันทึกผลลัพธ์เป็นไฟล์ .txt พร้อมระบุวันและเวลาที่บันทึก

ทดสอบการทำงานของส่วนติดต่อผู้ใช้: ตรวจสอบว่าปุ่มทุกปุ่มทำงานได้และผลลัพธ์แสดงผลถูกต้อง

ทดสอบการทำงานข้ามระบบปฏิบัติการ คอมไพล์โปรเจกต์สำหรับระบบปฏิบัติการ Windows ด้วยคำสั่ง `cargo tauri build --target windows` ทดสอบการทำงานทุกฟังก์ชันบน Windows: การจับภาพ (capture), การรู้จำข้อความ (OCR), การแปล (translation) และการคัดลอก/บันทึกผลลัพธ์ คอมไพล์โปรเจกต์สำหรับระบบปฏิบัติการ macOS ด้วยคำสั่ง `cargo tauri build --target macOS` ทดสอบการทำงานทุกฟังก์ชันบน macOS ด้วยขั้นตอนเดียวกันกับข้อ 2 คอมไพล์โปรเจกต์สำหรับระบบปฏิบัติการ Linux ด้วยคำสั่ง `cargo tauri build --target linux` ทดสอบการทำงานทุกฟังก์ชันบน Linux ด้วยขั้นตอนเดียวกันกับข้อ 2